Programming Fundamentals

Week 7 — Monday: Files and Persistence

Programming Fundamentals Team

SETU

2026-10-28

Today: Make Data Survive

Learning goals: explain persistence; read, write and append text; use `with open()`; recognise JSON; save and load a Python collection.

Remember Our CRUD Program

Our Dog Day Care list lived in memory:

```
dogs = [{"id": 1, "name": "Buddy", "age": 4}]
```

Close the program and start it again. **What happens to a dog we added?**

Remember Our CRUD Program

Our Dog Day Care list lived in memory:

```
dogs = [{"id": 1, "name": "Buddy", "age": 4}]
```

Close the program and start it again. **What happens to a dog we added?**

The list is recreated from the code. New records are lost.

What Is Persistence?

Persistence means keeping information after a program finishes.

Memory (RAM): temporary data while a program runs.

File: data saved on storage, available next time.

Think: a shopping list, game progress, or saved preferences.

A File Is Not a Python Variable

A text file stores **characters**. A Python variable can hold a string, number, list or dictionary.

We must decide **how** to represent our data when saving it.

Today: plain text first, then JSON for structured data.

Write Your First File

```
with open("message.txt", "w", encoding="utf-8") as file:
    file.write("Hello from Python!\n")
```

Run this in your project folder. Open `message.txt` and check its contents.

Predict: what happens if we run it again?

Why Use `with`?

`open()` gives us a file object.

`with` automatically closes the file when we leave the block, including if an error occurs.

Indentation matters: file operations belong inside the `with` block.

Three Useful File Modes

- `"w"` — **write**: create or overwrite a file
- `"a"` — **append**: add to the end
- `"r"` — **read**: open an existing file

Warning: `"w"` replaces the old contents. It does not add to them.

Prediction: Write or Append?

```
with open("notes.txt", "w") as f:
    f.write("First\n")
with open("notes.txt", "a") as f:
    f.write("Second\n")
```

Poll: what lines are in `notes.txt` now?

Prediction: Write or Append?

```
with open("notes.txt", "w") as f:
    f.write("First\n")
with open("notes.txt", "a") as f:
    f.write("Second\n")
```

Poll: what lines are in `notes.txt` now?

Answer: `First` and `Second`.

Read a Whole Text File

```
with open("notes.txt", "r", encoding="utf-8") as file:
    contents = file.read()
print(contents)
```

`read()` returns a **string**. The file must already exist.

Read One Line at a Time

```
with open("notes.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())
```

`strip()` removes the trailing newline for cleaner printing.

Discuss: why might a loop be useful for a long file?

Live Coding: A Simple Journal

Build a menu with two choices: **add a note** and **view notes**.

Pseudocode:

```
ASK user for a note
APPEND the note to journal.txt
OPEN journal.txt for reading
DISPLAY each saved line
```

What If the File Does Not Exist?

```
try:
    with open("journal.txt", "r") as file:
        print(file.read())
except FileNotFoundError:
    print("No journal entries yet")
```

A friendly message is better than a crash. We will study exceptions in more detail later.

Activity: Predict, Run, Explain

1. Write `"Monday"` using `"w"`.
2. Append `"Wednesday"` using `"a"`.
3. Read and display both lines.
4. Change the append mode to write mode and run again.

Why did the result change?

But What About Dictionaries?

Our Dog Day Care data is structured:

```
dogs = [
    {"id": 1, "name": "Buddy", "age": 4},
    {"id": 2, "name": "Luna", "age": 2}
]
```

We want to keep the **list of dictionaries**, not just print them as text.

Meet JSON

JSON stands for JavaScript Object Notation. It is a widely used text format for structured data.

Python list → JSON array

Python dictionary → JSON object

JSON is a file format, not a database or a Python class.

Save a List as JSON

```
import json

with open("dogs.json", "w", encoding="utf-8") as file:
    json.dump(dogs, file, indent=4)
```

`dump()` writes Python data as JSON. `indent=4` makes the file easier to read.

Load JSON Back into Python

```
import json

with open("dogs.json", "r", encoding="utf-8") as file:
    dogs = json.load(file)

print(dogs[0]["name"])
```

`load()` reads JSON and builds Python lists/dictionaries.

Check Your Understanding

Quick questions:

- Which mode would you use to add a line without erasing existing data?
- What does `with` help us do?
- What is the difference between `json.dump()` and `json.load()`?
- What happens if you read a file that does not exist?

Wednesday

On Wednesday we will use JSON to give our **PC Troubleshooter** a memory: its knowledge will live in a file, not just in Python code.